PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Actividad API Rest con Express, MySQL

Presentado a: Instructora Alexandra Muñoz

Por Aprendiz: **Nicolas Ruiz Colorado**

Ficha: 3312932

Competencia:

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **28** de **octubre** del año **2026**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Tabla de Contenido

API Rest con Express, MySQL 3

1. Investigación y preparación del entorno 3

1.1. ¿Qué es Node.js y que función cumple? 3

1.2. ¿Qué es Express y por qué se utiliza para desarrollar una API? 3

1.3. ¿Qué es una API REST? 3

1.4. ¿Qué función cumple CORS? 4

1.5. ¿Qué es MySQL y para qué sirve una base de datos? 4

1.6. ¿Qué información se necesita para establecer una conexión con MySQL en línea? 5

1.7. ¿Qué función cumple cada dependencia instalada en el proyecto? 5

2. Diseño de la base de datos 5

3. Construcción de la API 6

4. Revisión y análisis de errores 7

5. Pruebas funcionales 9

6. Documentación 11

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

API Rest con Express, MySQL

1. Investigación y preparación del entorno

1.1. ¿Qué es Node.js y que función cumple?

Node.js es un entorno de ejecución de código abierto y multiplataforma que permite ejecutar código JavaScript fuera del navegador web.

La función principal es permitir el desarrollo de backend, utilizando JavaScript, un modelo de entrada/salida, basado en eventos

- https://www.aluracursos.com/blog/node-js-que-es-como-funciona-este-entorno-de-ejecucion-de-javascript-y-una-guia-para-empezar
- https://profile.es/blog/que-es-nodejs/amp/

1.2. ¿Qué es Express y por qué se utiliza para desarrollar una API?

Es un framework web, que simplifica la creación de servidores, aplicaciones web y APIs; gestiona rutas sencillas, uso de middlewares, ahorro de código y flexibilidad

- https://www.codecademy.com/article/what-is-express-js?_x_tr_hist=true

1.3. ¿Qué es una API REST?

Es una interfaz de programación de aplicaciones que sigue los principios la arquitectura REST, que sirve para conectar aplicaciones móviles o sitios web con

servidores, por medio del protocolo HTTP y métodos del mismo como el (GET,POST,PUT, DELETE)

- https://www.ibm.com/mx-es/think/topics/rest-apis

1.4. ¿Qué función cumple CORS?

CORS es un mecanismo de seguridad implementado por los navegadores web que permite o restringe que una pagina web solicite recursos a un dominio, esquema o puerto diferente al que sirvió la pagina original;

Se realiza una petición → se espera la respuesta del servidor → navegador la verifica y la acepta o la bloquea si no cumple la condición mostrando la respuesta de error en el CORS

- https://aws.amazon.com/es/what-is/cross-origin-resource-sharing/

1.5. ¿Qué es MySQL y para qué sirve una base de datos?

Es un sistema de gestión de bases de datos relacionales de código abierto que organiza la información en tablas (compuestas en filas y columnas) que se relacionan entre si mediante claves

Una base de datos es un conjunto organizado de información recopilada y almacenada de forma electrónica en un sistema de información.

- https://www.unir.net/revista/ingenieria/mysql/

1.6. ¿Qué información se necesita para establecer una conexión con MySQL en línea?

Para poder realizar una conexión con MySQL se necesitan reunir los siguientes datos y requisitos principales como:

- Host o dirección ip
- Puerto
- Nombre de usuario
- Contraseña
- Nombre de la base de datos

https://dev.mysql.com/doc/refman/9.7/en/connecting.html

1.7. ¿Qué función cumple cada dependencia instalada en el proyecto?

2. Diseño de la base de datos

Campo	*Tipo*	*Tamaño*	*Tipo Key*
idProducto	Int		PK
Producto	Varchar	200	
Categoría	Varchar	100	
Precio	Int		
Stock	Int		

idProducto: decidí hacerlo con un tipo de dato int porque será un numero autoincremental

Producto: varchar porque es una cadena de texto

Categoría: varchar porque es una cadena de texto

Precio: int porque al ser números es la opción mas viable también podría usar un decimal, pero es un poco mas complejo de usar

Stock int porque es un valor de cantidad

```sql
create database tienda charset utf8mb4;
use tienda;
create table productos(
idProduvto int auto_increment primary key,
producto varchar(200) not null,
categoria varchar (100) not null,
precio int not null,
stock int not null
);

select * from productos;
```

3. **Construcción de la API**

4. Revisión y análisis de errores

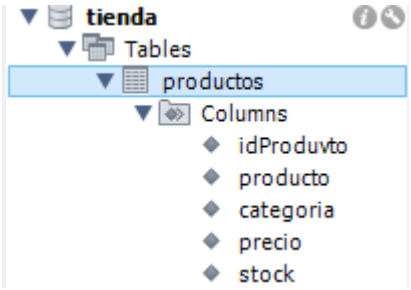
4.1. Configuración del middleware necesario para recibir solicitudes JSON

```
const cors = require('cors')
const express= require('express')
const app = express()
const PORT = 5000
app.use(express.json())
app.use(cors({
    origin:"http://localhost:5173"
}))
app.listen(PORT,()=>{
    console.log(`Servidor ejecutandose en el puerto ${PORT}`)
})
```

4.2. Conexión con la base de datos

```
const mysql=require('mysql2')
let conexion = mysql.createConnection({
    host:"127.0.0.1",
    user:"root",
    password:"050998",
    database:"tienda",
    port:"3306"
})
conexion.connect((error)=>{
    if(error){
        console.log("error de conexión base de datos")
    }
    else{
        console.log("Conexión realizada")
    }
})
```

4.3. Nombres de tablas y campos utilizados en las consultas

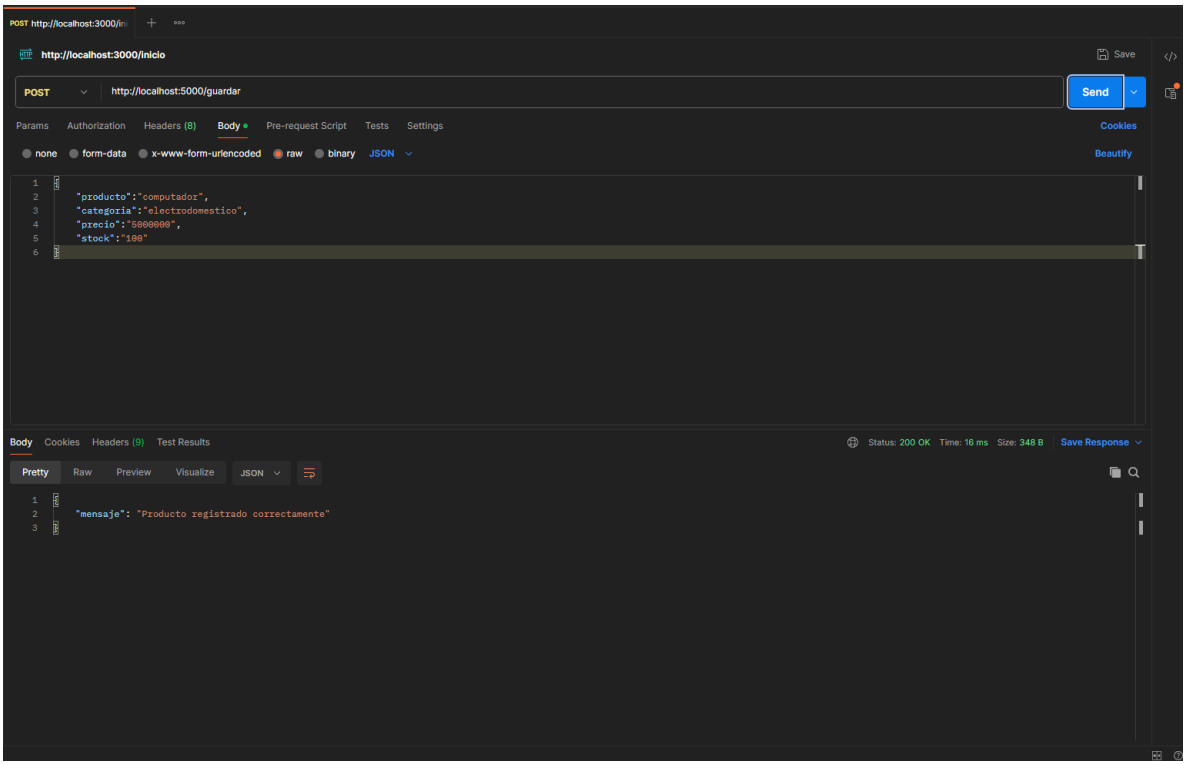



```
app.post('/guardar',(req,res)=>{
    const {producto,categoria,precio,stock} = req.body
    conexion.query("INSERT INTO productos
(producto,categoria,precio,stock) VALUES (?,?,?,?)",
        [producto,categoria,precio,stock],(error)=>{
            if (error){
                return res.json({
                    "mensaje":"Error al registrar el producto"
                })
            }
            else{
                return res.json({
                    "mensaje":"Producto registrado correctamente"
                })
            }
        }
    )
})
app.get('/productos',(req,res)=>{
    conexion.query("select * from productos",(error,resultado)=>{
        if (error){
            return res.status(500).json({
                "error":"Error al consultar el producto"
            })
        }
        res.status(200).json({
            productos:resultado
        })
    })
```

```
})
```

5. Pruebas funcionales

5.1. Registro exitoso de un producto

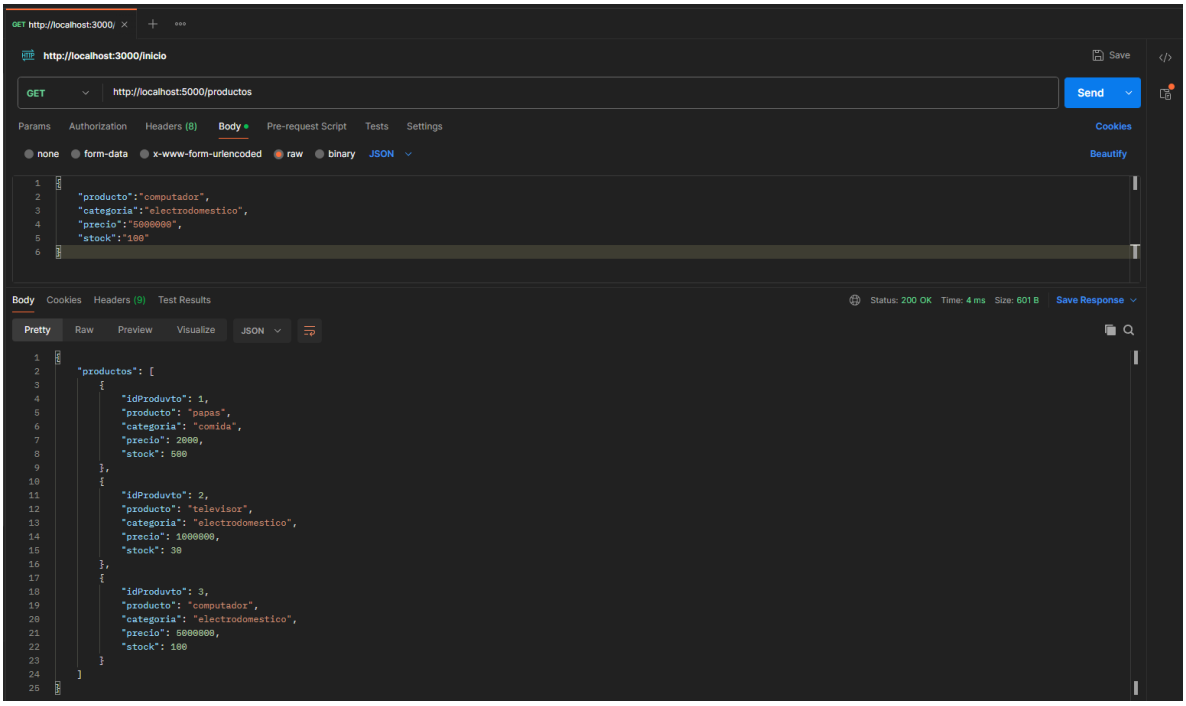


	idProduvto	producto	categoria	precio	stock
▶	1	papas	comida	2000	500
	2	televisor	electrodomestico	1000000	30
	3	computador	electrodomestico	5000000	100
*	NULL	NULL	NULL	NULL	NULL

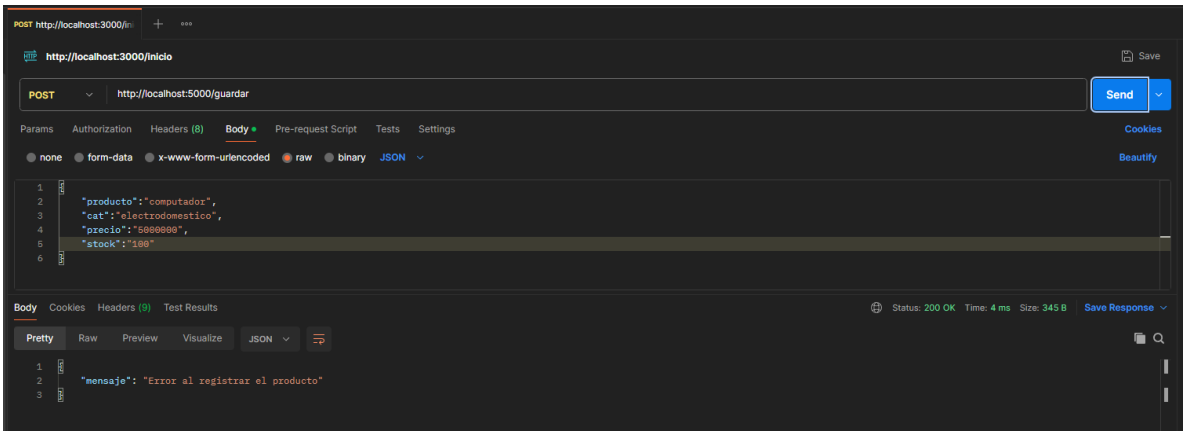
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

5.2. Consulta de productos registrados

	idProduvto	producto	categoria	precio	stock
▶	1	papas	comida	2000	500
	2	televisor	electrodomestico	1000000	30
	3	computador	electrodomestico	5000000	100
*	NULL	NULL	NULL	NULL	NULL



5.3. Envió de información incompleta



6. Documentación

6.1. Creación del proyecto

```
const cors = require('cors')
const express= require('express')
const app = express()
const PORT = 5000
const mysql=require('mysql2')

app.use(express.json())
app.use(cors({
    origin:"http://localhost:5173"
}))

let conexion = mysql.createConnection({
    host:"127.0.0.1",
    user:"root",
    password:"050998",
    database:"tienda",
    port:"3306"
})
```

```
conexion.connect((error)=>{
    if(error){
        console.log("error de conexion base de datos")
    }
    else{
        console.log("Conexion realizada")
    }
})
app.get('/prueba',(req,res)=>{
    res.json({
        "mensaje":"Esta es la conexion entre bs y el backend"
    })
})
app.post('/guardar',(req,res)=>{
    const {producto,categoria,precio,stock} = req.body
    conexion.query("INSERT INTO productos
(producto,categoria,precio,stock) VALUES (?,?,?,?)",
        [producto,categoria,precio,stock],(error)=>{
            if (error){
                return res.json({
                    "mensaje":"Error al registrar el producto"
                })
            }
            else{
                return res.json({
                    "mensaje":"Producto registrado correctamente"
                })
            }
        }
    )
})
app.get('/productos',(req,res)=>{
    conexion.query("select * from productos",(error,resultado)=>{
        if (error){
            return res.status(500).json({
                "error":"Error al consultar el producto"
            })
        }
        res.status(200).json({
            productos:resultado
```

```
        })
    })
})
app.listen(PORT,()=>{
    console.log(`Servidor ejecutandose en el puerto ${PORT}`)
})
```